

```
print("Processing command:", plaintext.decode())
```

but you should not make it actually edit/read files unless the assignment specifically asks for that.

For my python program I first created an encryption function that turns plaintext into ciphertext using AES-CTR encryption, A function that decrypts ciphertext into plaintext, a receiver function that displays ciphertext into plaintext, and a relay function that changes the action in the command using XOR on ciphertext. The design shows that even without the key and with just encryption itself it's vulnerable to a relay attack.

In my relay attack I changed the action READ into the action EDIT by calculating XOR differences between READ and EDIT and then applied these differences to the ciphertext bytes. I was then able to feed this new command into the receiver which after decryption displayed the new EDIT action instead of the READ. This was all done without being able to see the secret key showing how vulnerable encryption by itself can be without other layers. Furthermore, I was able to replay the original ciphertext two times and the command was processed.

The root cause of the issue is the integrity vulnerability with AES-CTR. This is because AES-CTR uses XOR to convert between plaintext and ciphertext. This means that edits to the ciphertext can create predictable outcomes to the plaintext. This means it is vulnerable to relay attacks that change the ciphertext without having to know the key.

This is solid, but I'd fix two terminology issues because they matter for the assignment.

First, what you demonstrated by changing READ to EDIT is better called a **bit-flipping/tampering attack**, not a relay attack. The assignment itself frames this as changing ciphertext with a relay function, but the security weakness is that CTR is malleable and lacks integrity. Second, your last sentence says "relay attacks that change the ciphertext," which could confuse that with the separate **replay attack** you also demonstrated. The homework's core point is specifically that confidentiality alone does not provide integrity or replay protection.  HW2-2

I'd revise your wording like this:

For my Python program, I first created an encryption function that turns plaintext into ciphertext using AES-CTR encryption, a function that decrypts ciphertext back into plaintext, a receiver function that displays the decrypted plaintext, and a relay function that modifies the action in the command by changing ciphertext bytes with XOR. The design shows that encryption by itself does not guarantee integrity or replay protection.

In my tampering experiment, I changed the action `READ` into `EDIT` by calculating the XOR differences between the bytes in `READ` and `EDIT`, then applying those differences to the corresponding ciphertext bytes. I was then able to send the modified ciphertext to the receiver, which decrypted it and displayed the new `EDIT` action instead of `READ`. This was done without knowing the secret key. I also replayed the original ciphertext twice, and the receiver processed the same command both times.

The root cause is that AES-CTR provides confidentiality but does not provide integrity. CTR works by XORing plaintext with a keystream, so controlled changes to ciphertext can cause predictable changes in the decrypted plaintext. In addition, the design does not use sequence numbers or other replay protection, so the receiver cannot tell when the same valid ciphertext has already been processed.

That version is more precise and matches what the assignment is asking you to explain.  HW2-2

For my python program I first created an encryption function that turns plaintext into ciphertext using AES-CTR encryption, A function that decrypts ciphertext into plaintext, a receiver function that displays ciphertext into plaintext, and a relay function that changes the action in the command using XOR on ciphertext. The design shows that even without the key and with just encryption itself it's vulnerable to a relay attack.

In my bit flipping attack I changed the action `READ` into the action `EDIT` by calculating XOR differences between `READ` and `EDIT` and then applied these differences to the ciphertext bytes. I was then able to feed this new command into the receiver which after decryption displayed the new `EDIT` action instead of the `READ`. This was all done without being able to see the secret key showing how vulnerable encryption by itself can be without other layers. Furthermore, I was able to replay the original ciphertext two times and the command was processed.

The root cause of the issue is the integrity vulnerability with AES-

CTR. This is because AES-CTR uses XOR to convert between plaintext and ciphertext. This means that edits to the ciphertext can create predictable outcomes to the plaintext. This means it is vulnerable to bit flipping attacks that change the ciphertext without having to know the key.

This is much better. The only thing I'd change is the phrase "**vulnerable to a relay attack**" in the first paragraph. Your actual attack is a **bit-flipping/tampering attack**, while the replay attack is the separate part where you resend the same ciphertext.

A cleaned-up version that stays very close to your wording would be:

For my Python program I first created an encryption function that turns plaintext into ciphertext using AES-CTR encryption, a function that decrypts ciphertext into plaintext, a receiver function that displays ciphertext as plaintext, and a relay function that changes the action in the command using XOR on the ciphertext. The design shows that even without knowing the key, encryption by itself is still vulnerable to ciphertext tampering.

In my bit-flipping attack I changed the action `READ` into the action `EDIT` by calculating the XOR differences between `READ` and `EDIT` and then applying these differences to the ciphertext bytes. I was then able to feed this new command into the receiver, which after decryption displayed the new `EDIT` action instead of `READ`. This was all done without being able to see the secret key, showing how vulnerable encryption by itself can be without other protection. Furthermore, I was able to replay the original ciphertext two times and the command was processed both times.

The root cause of the issue is the lack of integrity protection with AES-CTR. This is because AES-CTR uses XOR between the plaintext and a keystream. This means that edits to the ciphertext can create predictable changes in the plaintext after decryption. As a result, AES-CTR by itself is vulnerable to bit-flipping attacks because ciphertext can be modified without knowing the key. The design also has no replay protection, so the receiver cannot tell when the same ciphertext has already been processed.

That last sentence is worth adding because the assignment asks you to explain both the integrity problem **and** the replay problem.  HW2-2

How do i start task 2

Start Task 2 by thinking of it as **three smaller problems**, not one giant handshake:

1. create long-term RSA identity keys for the gateway and node,
2. create fresh ephemeral Diffie-Hellman values + nonces for one session,
3. build/sign/verify the same transcript, then derive session keys.

That is exactly the structure the assignment describes.



First, make sure you already generated the required DH group file from Task 0:

</> Bash

```
cd ~/csce465-agentsec/hw2
openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
```

Then verify it:

</> Bash

```
openssl dhparam -in ffdhe3072.pem -text -noout | head -3
```

You should see that it is a 3072-bit `ffdhe3072` group.



Then create your Task 2 file:

</> Bash

```
nano handshake.py
```

For now, I would only start with the imports and the first two helper functions.

</> Python

```
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding, dh
import os
```

The first thing you need is a function for the long-term RSA signing key:

</> Python

```
def generate_rsa_key():
    return rsa.generate_private_key(
        public_exponent=65537,
        key_size=3072
    )
```

Why?

The assignment says each party has a **long-term 3072-bit RSA signing key** and uses RSA-PSS with SHA-256.  HW2-2

So conceptually:

</> Plain text

Gateway:

- permanent RSA private key
- permanent RSA public key

Node:

- permanent RSA private key
- permanent RSA public key

These are for **authentication**, not for encrypting the messages. The assignment explicitly says the RSA keys authenticate the ephemeral exchange and are not used to encrypt application records.  HW2-2

Next, load the Diffie–Hellman parameters from the file:

</> Python

```
def load_dh_parameters():  
    with open("ffdhe3072.pem", "rb") as f:  
        return serialization.load_pem_parameters(f.read())
```

Then in `main()` :

</> Python

```
def main():  
    gateway_rsa = generate_rsa_key()  
    node_rsa = generate_rsa_key()  
  
    dh_parameters = load_dh_parameters()  
  
    gateway_dh = dh_parameters.generate_private_key()  
    node_dh = dh_parameters.generate_private_key()  
  
    gateway_nonce = os.urandom(16)  
    node_nonce = os.urandom(16)
```

```
print("Gateway RSA key created")
print("Node RSA key created")
print("Gateway DH key created")
print("Node DH key created")
print("Gateway nonce:", gateway_nonce.hex())
print("Node nonce:", node_nonce.hex())
```

Run:

</> Bash

```
python handshake.py
```

At this first checkpoint, you are only trying to prove:

</> Plain text

I have:

- gateway RSA key
- node RSA key
- gateway ephemeral DH key
- node ephemeral DH key
- gateway 16-byte nonce
- node 16-byte nonce

That corresponds to the first two substantive Task 2 requirements: long-term RSA keys, and fresh DH private values plus 16-byte nonces for each session.  HW2-2

The next step after this is the most important part: building the **canonical transcript** with these fields:

</> Plain text

```
CSCE465-HS-v2
ffdhe3072
gateway identity
node identity
gateway DH public value
node DH public value
gateway nonce
node nonce
```

Every field must be encoded as:

</> Plain text

4-byte big-endian length || field bytes

and the DH public values must be represented as exactly **384-byte big-endian values**.  HW2-2

So I would not jump into signing yet. Get this first checkpoint running, then the next thing we should build together is the transcript encoder.

Yesterday 8:01 PM

```
def load_dh_parameters():
    with open("ffdhe3072.pem", "rb") as f:
        return serialization.load_pem_parameters(f.read()) this is written
        weirdly can return not be in a separate line
```

Yes. You can absolutely split it up so it reads more clearly.

Instead of:

</> Python

```
def load_dh_parameters():
    with open("ffdhe3072.pem", "rb") as f:
        return serialization.load_pem_parameters(f.read())
```

you can write:

</> Python

```
def load_dh_parameters():
    with open("ffdhe3072.pem", "rb") as f:
        data = f.read()

    parameters = serialization.load_pem_parameters(data)

    return parameters
```

This does the exact same thing, but it's easier to understand.

Step-by-step:

```
</> Python
```

```
with open("ffdhe3072.pem", "rb") as f:
```

opens the file in binary read mode.

```
</> Python
```

```
data = f.read()
```

reads the contents of the PEM file into `data` .

Then:

```
</> Python
```

```
parameters = serialization.load_pem_parameters(data)
```

takes those raw bytes and converts them into a Diffie–Hellman parameters object that the `cryptography` library can use.

Finally:

```
</> Python
```

```
return parameters
```

returns that object from the function.

So this version is perfectly fine and probably better for learning because each step is visible.

what is nonce

A **nonce** is a random value that is meant to be used once for a particular protocol run.

In your Task 2 code, you have:

```
</> Python
```

```
gateway_nonce = os.urandom(16)
node_nonce = os.urandom(16)
```

That gives each side a fresh 16-byte random value for the current handshake, which is exactly what the assignment requires.  HW2-2

You can think of a nonce like a one-time challenge or session marker. Even if the same gateway and node communicate again later, they should generate new nonces, so the new handshake transcript is different from the old one.

That matters for **replay resistance**. Suppose an attacker recorded an old valid handshake. If the new session uses fresh nonces, the old signed transcript no longer matches the new handshake, so replaying the old messages should fail.

So:

</> Plain text

Session 1:

gateway nonce = random value A

node nonce = random value B

Session 2:

gateway nonce = random value C

node nonce = random value D

The assignment includes both nonces in the signed transcript, so they become part of what each side authenticates.  HW2-2

A short definition you could remember is:

A nonce is a fresh one-time value used to make each protocol session unique and help prevent replay attacks.

Gateway RSA key created

Node RSA key created

Gateway DH key created

Node DH key created

Gateway nonce: 25f71013d561443ecb9504ed59463ffa

Node nonce: 0eb0faaf9fc7f7348f0718ae7ea24f41

Perfect — that means your first Task 2 checkpoint is working.

You now have:

- a long-term RSA key for the gateway
- a long-term RSA key for the node
- a fresh Diffie–Hellman private key for each side
- a fresh 16-byte nonce for each side

That matches the first major requirements of Task 2.  HW2-2

The next thing to build is the **canonical transcript**. Your assignment says it needs to contain, in order:

`</>` Plain text

```
CSCE465-HS-v2
ffdhe3072
gateway identity
node identity
gateway DH public value
node DH public value
gateway nonce
node nonce
```

and every field has to be encoded as:

`</>` Plain text

```
4-byte big-endian length || field
```

The DH public values specifically need to be encoded as exactly **384-byte big-endian values**.  HW2-2

So your next step is to learn how to:

1. get the numeric DH public values,
2. turn them into 384-byte strings,
3. write a helper that length-prefixes each field.

That's the part I'd do next.

which steps have i done so far

So far, for **Task 2**, you've completed the beginning of the handshake setup.

You've done these parts:

- Generated/loaded the `ffdhe3072` Diffie–Hellman group file from Task 0.

- Created a **3072-bit RSA private key** for the gateway.
- Created a **3072-bit RSA private key** for the node.
- Generated a fresh **ephemeral Diffie–Hellman private key** for the gateway.
- Generated a fresh **ephemeral Diffie–Hellman private key** for the node.
- Generated a fresh **16-byte nonce** for each side.

That covers the first two main requirements of Task 2.  HW2-2

You have **not done yet**:

- extracting the DH **public values**
- encoding each DH public value as exactly **384 bytes**
- building the canonical transcript
- adding the 4-byte big-endian length prefix to every transcript field
- hashing the transcript with SHA-256
- signing `role || SHA-256(transcript)` with RSA-PSS
- verifying the peer signature and identity
- computing the shared DH secret `Z`
- deriving `K_master`
- deriving the 4 directional keys and `session_id`
- adding rejection checks for changed nonces/public values, malformed transcript, wrong identity, or reflection

Those are the remaining Task 2 requirements.  HW2-2

So you're roughly at:

</> Plain text

Task 2

- ☒ RSA identity keys
- ☒ ephemeral DH private keys
- ☒ nonces
- ☐ DH public values
- ☐ canonical transcript
- ☐ signatures + verification
- ☐ shared secret
- ☐ key derivation
- ☐ rejection tests

Next, the cleanest step is **getting the gateway and node DH public values and converting them to 384-byte big-endian form**.

walk me through the next steps

Yes. The next part is where Task 2 starts turning your generated keys/nonces into an actual authenticated handshake.

You already have:

</> Plain text

```
gateway RSA key
node RSA key
gateway DH private key
node DH private key
gateway nonce
node nonce
```

The assignment now wants you to build one canonical transcript containing the protocol label, group ID, both identities, both ephemeral DH public values, and both nonces. Every field must be length-prefixed with a 4-byte big-endian integer.  HW2-2

Step 1: Get the DH public values

You currently have DH private keys:

</> Python

```
gateway_dh
node_dh
```

Each private key has a corresponding public key.

Add:

</> Python

```
gateway_public_key = gateway_dh.public_key()
node_public_key = node_dh.public_key()
```

Now these are public-key objects, but the assignment wants the actual DH public value as exactly **384 bytes**.  HW2-2

You can get the integer public value like this:

</> Python

```
gateway_public_number = gateway_public_key.public_numbers().y
node_public_number = node_public_key.public_numbers().y
```

The `.y` is the actual numeric DH public value.

Now convert each integer to 384 bytes:

</> Python

```
gateway_public_bytes = gateway_public_number.to_bytes(384, "big")
node_public_bytes = node_public_number.to_bytes(384, "big")
```

Why 384?

</> Plain text

3072 bits / 8 = 384 bytes

And "big" means big-endian byte order, exactly as the assignment requires.

Step 2: Define the fixed transcript fields

You also need these:

</> Python

```
protocol_label = b"CSCE465-HS-v2"
group_id = b"ffdhe3072"

gateway_identity = b"gateway"
node_identity = b"node"
```

The assignment says both identities must be included.

 HW2-2

Step 3: Make a helper for length-prefixing

Each field must look like:

</> Plain text

4-byte length || actual field

So create:

</> Python

```
def encode_field(field):  
    length = len(field)  
    length_bytes = length.to_bytes(4, "big")  
    return length_bytes + field
```

For example:

</> Python

```
encode_field(b"node")
```

would conceptually become:

</> Plain text

00000004 || node

because "node" is 4 bytes long.

This prevents ambiguity when parsing the transcript later. The assignment specifically says plain concatenation is not acceptable.  HW2-2

Step 4: Build the canonical transcript

Now assemble everything in the required order:

</> Python

```
transcript = (  
    encode_field(protocol_label)  
    + encode_field(group_id)  
    + encode_field(gateway_identity)
```

```
+ encode_field(node_identity)
+ encode_field(gateway_public_bytes)
+ encode_field(node_public_bytes)
+ encode_field(gateway_nonce)
+ encode_field(node_nonce)
)
```

Now `transcript` is just one long `bytes` object.

You can test it:

</> Python

```
print("Transcript length:", len(transcript))
```

You do not need to print the whole thing unless you want to inspect it.

Step 5: Hash the transcript

The assignment defines:

</> Plain text

```
TH = SHA-256(transcript)
```

 HW2-2

Using `cryptography` :

</> Python

```
digest = hashes.Hash(hashes.SHA256())
digest.update(transcript)
transcript_hash = digest.finalize()
```

Then:

</> Python

```
print("Transcript hash:", transcript_hash.hex())
```

Both gateway and node should independently compute the exact same transcript hash.

Step 6: Sign the transcript hash

The assignment says each party signs:

</> Plain text

```
role || SHA-256(transcript)
```

using RSA-PSS with SHA-256.  HW2-2

You can make a helper:

</> Python

```
def sign_transcript(private_key, role, transcript_hash):  
    message = role + transcript_hash  
  
    signature = private_key.sign(  
        message,  
        padding.PSS(  
            mgf=padding.MGF1(hashes.SHA256()),  
            salt_length=padding.PSS.MAX_LENGTH  
        ),  
        hashes.SHA256()  
    )  
  
    return signature
```

Then:

</> Python

```
gateway_signature = sign_transcript(  
    gateway_rsa,  
    b"gateway",  
    transcript_hash  
)  
  
node_signature = sign_transcript(  
    node_rsa,  
    b"node",
```

```
        transcript_hash
    )
```

The role matters because it helps prevent one side's signed message from simply being reflected back as if it came from the other role.

Step 7: Verify the signatures

Now each side must verify the other side.

A helper could look like:

</> Python

```
def verify_signature(public_key, role, transcript_hash, signature):
    message = role + transcript_hash

    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

Then:

</> Python

```
gateway_rsa_public = gateway_rsa.public_key()
node_rsa_public = node_rsa.public_key()
```

Gateway verifies node:

</> Python

```
verify_signature(
    node_rsa_public,
    b"node",
    transcript_hash,
```

```
node_signature
)
```

Node verifies gateway:

</> Python

```
verify_signature(
    gateway_rsa_public,
    b"gateway",
    transcript_hash,
    gateway_signature
)
```

If verification succeeds, `verify()` returns normally.

If the signature is invalid, it throws an exception.

That satisfies the authentication part of the handshake.  HW2-2

Step 8: Compute the DH shared secret

Now comes the actual Diffie–Hellman key agreement.

Gateway computes:

</> Python

```
gateway_shared = gateway_dh.exchange(node_public_key)
```

Node computes:

</> Python

```
node_shared = node_dh.exchange(gateway_public_key)
```

These should be equal:

</> Python

```
print(gateway_shared == node_shared)
```

You should get:

</> Plain text

True

That is the core Diffie–Hellman property:

</> Plain text

gateway independently computes Z
node independently computes Z
both get the same result

The assignment calls this shared secret Z and wants it represented as exactly 384 bytes.



HW2-2

Depending on the library output, I would inspect:

</> Python

```
print(len(gateway_shared))
```

and make sure it is 384 bytes for this group.

Step 9: Derive K_{master}

The assignment gives you the exact formula:

</> Plain text

```
K_master =  
SHA-256("CSCE465-KDF-v1" || Z || TH)
```

 HW2-2

So:

</> Python

```
digest = hashes.Hash(hashes.SHA256())

digest.update(b"CSCE465-KDF-v1")
digest.update(gateway_shared)
digest.update(transcript_hash)

k_master = digest.finalize()
```

Then:

</> Python

```
print("Master key:", k_master.hex())
```

You should do the same from the node side eventually and confirm:

</> Plain text

```
gateway K_master == node K_master
```

Step 10: Derive the four directional keys

The homework gives you exact labels, so use them exactly.  HW2-2

First import:

</> Python

```
from cryptography.hazmat.primitives import hmac
```

Then make a helper:

</> Python

```
def derive_key(k_master, label, transcript_hash):
    h = hmac.HMAC(k_master, hashes.SHA256())
    h.update(label + transcript_hash)
    return h.finalize()
```

Then:

</> Python

```
k_g2n_enc = derive_key(
    k_master,
    b"gateway-to-node encryption",
    transcript_hash
)

k_g2n_mac = derive_key(
    k_master,
    b"gateway-to-node MAC",
    transcript_hash
)

k_n2g_enc = derive_key(
    k_master,
    b"node-to-gateway encryption",
    transcript_hash
)

k_n2g_mac = derive_key(
    k_master,
    b"node-to-gateway MAC",
```

Now you have separate keys for:

</> Plain text

```
gateway -> node encryption
gateway -> node MAC

node -> gateway encryption
node -> gateway MAC
```

That is key separation.

Step 11: Derive the session ID

The assignment also requires:

</> Plain text

```
session_id =
    first 8 bytes of
    HMAC-SHA-256(
        K_master,
```

```
"session identifier" || TH
)
```

 HW2-2

You can do:

</> Python

```
session_full = derive_key(
    k_master,
    b"session identifier",
    transcript_hash
)

session_id = session_full[:8]
```

Then:

</> Python

```
print("Session ID:", session_id.hex())
```

At that point, Task 2's successful handshake path is mostly built.

Your program flow will roughly be:

</> Plain text

```
generate RSA keys
    ↓
load ffdhe3072 parameters
    ↓
generate ephemeral DH keys
    ↓
generate nonces
    ↓
get DH public values
    ↓
encode public values as 384 bytes
    ↓
build canonical transcript
    ↓
```

```
SHA-256 transcript
  ↓
gateway signs
node signs
  ↓
verify signatures
```

After this succeeds, the remaining part of Task 2 is adding the **failure cases**: reject invalid signatures, changed nonce/public value, malformed transcript, unexpected identity, and reflected handshake messages.  HW2-2

I would implement only through **Step 4 first**—DH public values + transcript encoding—and run it before adding signing. That will make debugging much easier.

is `gateway_public_key` not `gateway_rsa_public`

They're two different public keys for two different purposes.

`gateway_public_key` in the code I showed referred to the **Diffie–Hellman public key**:

 Python

```
gateway_public_key = gateway_dh.public_key()
```

That public key is used for the DH key exchange.

`gateway_rsa_public` is the **RSA public key**:

 Python

```
gateway_rsa_public = gateway_rsa.public_key()
```

That one is used to verify RSA-PSS signatures.

So it may be clearer to rename them like this:

 Python

```
gateway_dh_public = gateway_dh.public_key()
node_dh_public = node_dh.public_key()
```